

Jeu iOS : Slide or Die

Publier un jeu sur l'App Store

I. Contexte et objectif.....	1
II. Stack technique.....	1
III. Résultat.....	2
IV. Conclusion... ?.....	3

I. Contexte et objectif

Contexte: en juin 2025 après avoir fait une pause d'un petit mois sur mon premier gros projet iOS, je me rends compte que ce premier projet était un peu optimiste. En effet, en regardant de nouveau ce que j'avais produit, il y avait beaucoup de bonnes choses, mais aussi quantité de choses à refaire. C'est assez logique comme j'apprends énormément sur ces projets, mon niveau croît rapidement. Mais le code produit plusieurs mois auparavant ne correspond plus forcément à mes standards. Fort de ce constat, je décide de développer un petit jeu plus simple et de le publier sur l'App Store. Cette approche me permettra de préparer efficacement la sortie du projet principal, en anticipant les éventuels obstacles techniques ou administratifs liés à la distribution mobile.

Évidemment si j'en parle 8 mois plus tard, c'est que le projet a pris une autre dimension qu'un simple jeu à publier sur l'App Store.

Objectif initial : créer et publier un premier jeu plutôt simple sur iOS.

Objectif actuel : créer et publier un jeu multijoueur complet sur iOS.

II. Stack technique

Pour ce projet, j'utilise Swift, le langage natif d'iOS, en m'appuyant principalement sur SwiftUI pour l'interface utilisateur et SpriteKit pour la partie graphique et la logique du jeu. Ces deux frameworks sont essentiels pour concevoir un jeu fluide et bien intégré à l'environnement Apple. J'utilise aussi Google AdMob pour la gestion des publicités sur l'App. Les différentes tailles d'écran sont gérées, de même que différentes langues (et en rajouter est très simple).

Pour gérer les données des joueurs, j'ai un serveur Quarkus qui se connecte via Hibernate à une base de données PostgreSQL. Quarkus fonctionne aussi avec Keycloak car c'est "simple" et efficace.

Pour gérer les parties multijoueur, je développe mon propre moteur de jeu en C++. Je crée quelque chose de franchement robuste là dessus (CRTP, ECS, SoA, que du compile-time ou presque sur l'update principal, maillage spatial...). J'utilise des WebSockets pour communiquer entre iOS et C++, avec des clés générées par Quarkus avec l'autorisation de Keycloak.

Le tout bien caché derrière un Nginx, et sous Docker.

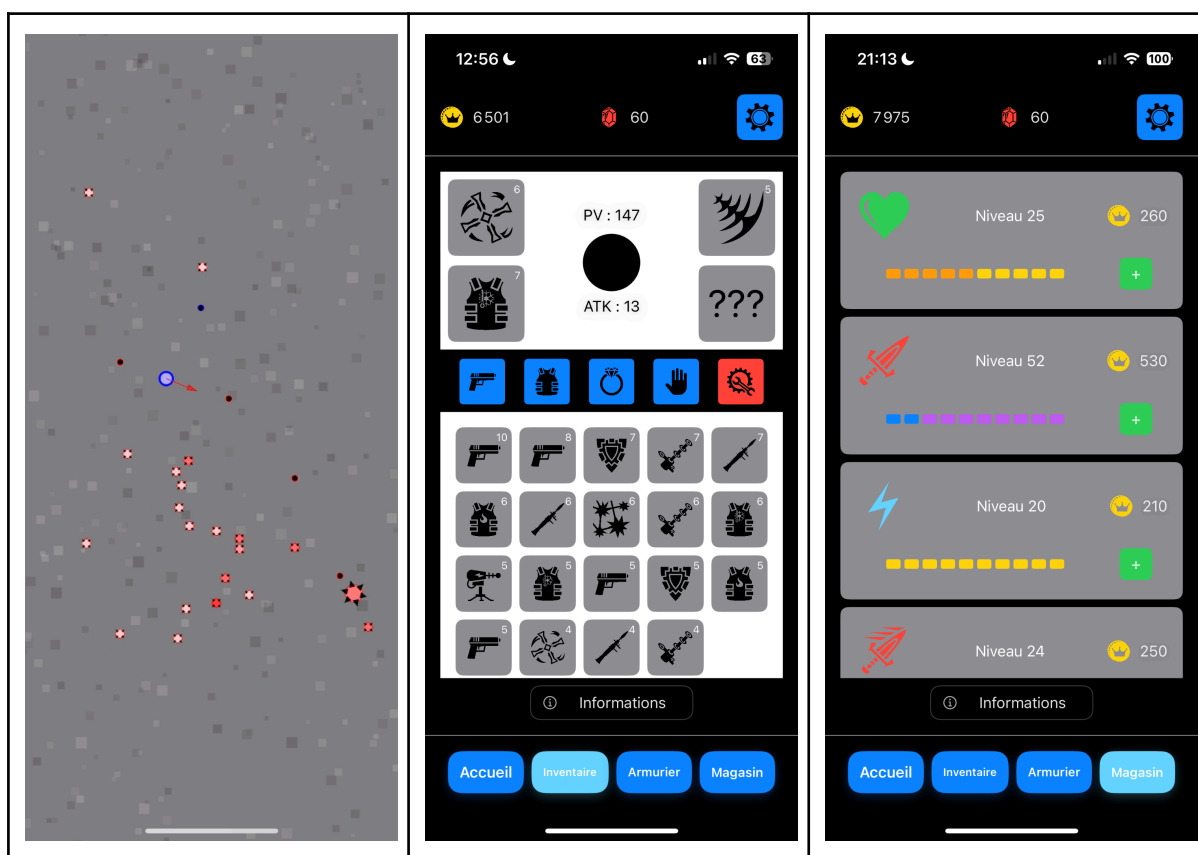
III. Résultat

Le plus simple pour voir le résultat de mon jeu, c'est encore d'en tester la bêta ! Voici le lien pour y accéder : <https://testflight.apple.com/join/RSa4Zagh>.

Sinon, voici l'état du jeu en quelques lignes :

- Interface graphique simpliste mais claire
- Un système de progression efficace
- Deux modes de jeu : mode infini (classement par score) et mode niveau (classement par meilleur niveau complété)
- Multijoueur fonctionnel : si deux joueurs rejoignent une partie en même temps, ils sont ensemble pour tenter de survivre le plus longtemps possible
- 6 armes, 3 armures, 2 modules à collectionner (débloquable avec la progression dans le jeu) et bien plus encore

Voici trois petites captures d'écran, même si le mieux pour comprendre ce que j'ai créé est de tester !



IV. Conclusion... ?

Ce qu'il me reste à créer pour publier le jeu c'est le quatrième emplacement d'équipement et les sons. Après ça, j'aurai un jeu "complet" qui embarque tout ce que je voulais y mettre. Du tout début du projet de fin 2024 à maintenant, j'ai énormément appris tant en développement pur qu'en gestion de projet.

Je vais bien sûr découvrir encore beaucoup de choses pour publier réellement mon jeu sur l'App Store. J'ai pour objectif de publier le jeu fin février ; cela peut sembler optimiste mais après tout pourquoi pas.

Ce qui est certain c'est que suite à ça je vais continuer à travailler sur ce jeu, tant sur la création de nouvelles fonctionnalités que sur sa maintenance. Travailler sur un jeu et le maintenir sont deux choses (j'imagine) très différentes, donc je vais là-dessus pouvoir apprendre beaucoup de choses.